

Cyber Security Lab (Ethical Hacking)

ICMP Redirect Attacks

Task 1: Setup the Network

Objective

The objective of this task is to configure the virtual network environment and verify that all devices can communicate correctly both within the same subnet and across different subnets. This step is essential to ensure that the network infrastructure is properly set up before performing any attack.

Network Topology

The laboratory environment consists of two subnets:

- Local network: 10.9.0.0/24
- Remote network: 192.168.60.0/24

The main devices involved are:

- Victim: 10.9.0.5
- Attacker: 10.9.0.105
- Malicious Router: 10.9.0.111
- Router: 10.9.0.11
- Servers: 192.168.60.5, 192.168.60.6

The communication between subnets is performed through the router.

Network Deployment

The network was deployed using Docker with the following command:

```
sudo docker compose up
```

This command automatically creates the containers, assigns IP addresses, and connects all devices according to the configuration defined in the *docker-compose.yml* file.

Connectivity Verification

Local Network Connectivity

From the victim machine, connectivity within the local subnet was tested:

```
ping 10.9.0.11 # router
ping 10.9.0.105 # attacker
ping 10.9.0.111 # malicious router
```

All devices responded successfully, confirming that communication within the same subnet works correctly without requiring a router.

Remote Network Connectivity

From the victim machine, connectivity to the remote network was tested:

```
ping 192.168.60.5
ping 192.168.60.6
```

The responses confirmed that communication between subnets is working correctly via the router.

```
05 csl-router
linuxuser@linuxuser-VirtualBox:~$ sudo docker exec -it f57c0434b36f bash
csl-victim> ping 10.9.0.11
PING 10.9.0.11 (10.9.0.11) 56(84) bytes of data:
64 bytes from 10.9.0.11: icmp_seq=1 ttl=64 time=1.41 ms
64 bytes from 10.9.0.11: icmp_seq=2 ttl=64 time=0.090 ms
64 bytes from 10.9.0.11: icmp_seq=3 ttl=64 time=0.064 ms
^C
--- 10.9.0.11 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2009ms
rtt min/avg/max/mdev = 0.064/0.520/1.408/0.627 ms
csl-victim> ping 10.9.0.105
PING 10.9.0.105 (10.9.0.105) 56(84) bytes of data:
64 bytes from 10.9.0.105: icmp_seq=1 ttl=64 time=0.453 ms
64 bytes from 10.9.0.105: icmp_seq=2 ttl=64 time=0.068 ms
64 bytes from 10.9.0.105: icmp_seq=3 ttl=64 time=0.100 ms
^C
--- 10.9.0.105 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2051ms
rtt min/avg/max/mdev = 0.068/0.207/0.453/0.174 ms
csl-victim> ping 10.9.0.111
PING 10.9.0.111 (10.9.0.111) 56(84) bytes of data:
64 bytes from 10.9.0.111: icmp_seq=1 ttl=64 time=0.598 ms
64 bytes from 10.9.0.111: icmp_seq=2 ttl=64 time=0.065 ms
64 bytes from 10.9.0.111: icmp_seq=3 ttl=64 time=0.080 ms
^C
--- 10.9.0.111 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2021ms
rtt min/avg/max/mdev = 0.065/0.247/0.598/0.247 ms
csl-victim> ping 192.168.60.5
PING 192.168.60.5 (192.168.60.5) 56(84) bytes of data:
64 bytes from 192.168.60.5: icmp_seq=1 ttl=63 time=0.409 ms
64 bytes from 192.168.60.5: icmp_seq=2 ttl=63 time=0.112 ms
64 bytes from 192.168.60.5: icmp_seq=3 ttl=63 time=0.118 ms
^C
--- 192.168.60.5 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2034ms
rtt min/avg/max/mdev = 0.112/0.213/0.409/0.138 ms
csl-victim> ping 192.168.60.6
PING 192.168.60.6 (192.168.60.6) 56(84) bytes of data:
64 bytes from 192.168.60.6: icmp_seq=1 ttl=63 time=0.425 ms
64 bytes from 192.168.60.6: icmp_seq=2 ttl=63 time=0.137 ms
64 bytes from 192.168.60.6: icmp_seq=3 ttl=63 time=0.143 ms
^C
--- 192.168.60.6 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2027ms
rtt min/avg/max/mdev = 0.137/0.235/0.425/0.134 ms
csl-victim>
```

Routing Table Analysis

The routing table was inspected using:

ip route

Output:

```
csl-victim> ip route
default via 10.9.0.1 dev eth0
10.9.0.0/24 dev eth0 proto kernel scope link src 10.9.0.5
192.168.60.0/24 via 10.9.0.11 dev eth0
csl-victim>
```

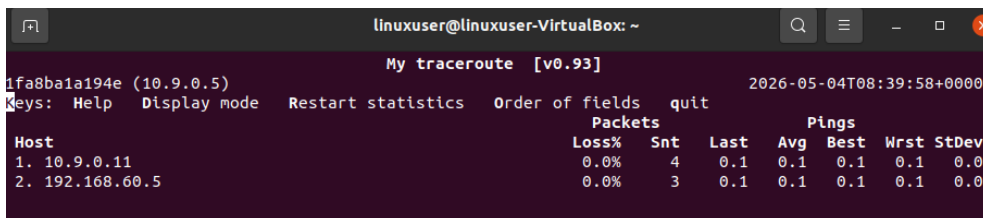
This indicates that traffic destined for the remote network is forwarded to the router at 10.9.0.11.

Path Verification

The path taken by packets was analyzed using:

mtr -n 192.168.60.5

Result:



```
linuxuser@linuxuser-VirtualBox: ~
My traceroute [v0.93]
2026-05-04T08:39:58+0000
Keys: Help Display mode Restart statistics Order of fields quit
Host      Loss% Snt  Last Avg Best Wrst StDev
1. 10.9.0.11 0.0% 4    0.1 0.1 0.1 0.1 0.0
2. 192.168.60.5 0.0% 3    0.1 0.1 0.1 0.1 0.0
```

This confirms that traffic is correctly routed through the router when accessing a different subnet.

ARP Table Inspection

The ARP table was examined using:

arp -a

This command displays the mapping between IP addresses and MAC addresses of devices in the local network, confirming that address resolution is functioning correctly.

Conclusion

The network is correctly configured and operational. Devices within the same subnet can communicate directly, while communication between different subnets is correctly handled by the router. The routing table, path analysis, and ARP resolution all confirm that the infrastructure is functioning as expected.

Task 2: Understanding ICMP Redirect Messages

Objective

The objective of this task is to analyze whether a victim machine accepts ICMP redirect messages and to understand how these messages can be exploited to manipulate routing behavior. To achieve this, a script was developed using the Scapy library to generate spoofed ICMP redirect packets and observe their effect on the victim.

Methodology

Step 1 – Generate Network Traffic

ICMP redirect messages are only processed when there is active communication. Therefore, traffic was generated from the victim to a remote host:

```
ping 192.168.60.5
```

Step 2 – Execute the ICMP Redirect Attack

The spoofed ICMP redirect messages were sent from the attacker using a custom Python script:

```
python3 icmp_spoof.py  
or  
while true; do python3 icmp_spoof.py; done
```

The script impersonates the legitimate router and instructs the victim to use an alternative gateway (malicious router).

Step 3 – Verify the Routing Behavior

To verify whether the attack was successful, the path of the packets was analyzed using:

mtr -n 192.168.60.5

My traceroute [v0.93]								
1fa8ba1a194e (10.9.0.5)			2026-05-04T08:50:20+0000					
Keys:	Help	Display mode	Restart statistics	Order of fields	quit			
Host	Packets			Pings				
	Loss%	Snt	Last	Avg	Best	Wrst	StDev	
1. 10.9.0.111	0.0%	4	0.1	0.1	0.1	0.1	0.0	
2. 10.9.0.11	0.0%	4	1.7	0.9	0.5	1.7	0.6	
3. 192.168.60.5	0.0%	3	0.5	0.5	0.5	0.6	0.0	

Step 4 – Interpretation

- If traffic is routed through 10.9.0.111 (malicious router), the attack is successful.
- If traffic continues through 10.9.0.11 (legitimate router), the attack fails.

Important Concept

ICMP redirect messages do not modify the routing table (`ip route`). Instead, they affect routing decisions dynamically through the routing cache.

- `ip route` remains unchanged
- `mtr` reflects the actual path taken by packets

Scenario Analysis

Scenario 1: Attacker, victim, and malicious router in the same subnet

All devices are located within the local network 10.9.0.0/24.

Result:

The attack is successful.

Explanation:

The ICMP redirect message appears to come from a valid router and suggests a reachable alternative gateway within the same subnet. The victim accepts the message and updates its routing behavior accordingly.

Evidence:

mtr shows that traffic is routed through 10.9.0.111.

Scenario 2: Malicious router located in a different subnet

The attacker attempts to redirect traffic through a router located outside the local network.

Result:

The attack is not successful.

Explanation:

The victim cannot directly reach the proposed gateway. Since the malicious router is not in the same subnet, ARP resolution fails and the redirect message is ignored.

Scenario 3: Non-existing router

The attacker specifies a gateway IP address that does not correspond to any existing device.

Result:

The attack is not successful.

Explanation:

The victim attempts to resolve the MAC address of the gateway but receives no response. As a result, the route cannot be established and the redirect is discarded.

Scenario 4: Attacker located in a different subnet

The attacker sends ICMP redirect messages from an IP address outside the victim's subnet.

Result:

The attack is not successful.

Explanation:

ICMP redirect messages must originate from a router within the same subnet. Messages coming from external networks are considered invalid and are rejected by the victim.

Routing Table vs Routing Cache

Routing Table	Routing Cache
Contains static routing information	Stores temporary routing decisions
Defines default paths	Is influenced by ICMP redirect messages
Is not modified by ICMP redirect messages	Updates dynamically based on traffic conditions

Key Observations

- ICMP redirect messages are only accepted under strict conditions
- The alternative gateway must be reachable
- The message must originate from a valid router in the same subnet
- The attack effect is temporary and depends on the routing cache
- Active traffic is required for the redirect to take effect

Conclusion

The ICMP redirect attack is effective only when specific conditions are met. The attacker must operate within the same subnet as the victim, and the malicious router must be reachable. Additionally, the spoofed message must appear to originate from a legitimate router.

If these conditions are not satisfied, the victim system ignores the redirect message due to built-in validation mechanisms. This demonstrates that while ICMP redirect can be exploited, modern systems include safeguards that limit its effectiveness.

Task 3: Establish Yourself as the Man in the Middle (Eavesdropper)

Objective

The objective of this task is to exploit ICMP redirect messages in order to position the malicious router as a Man-in-the-Middle (MITM) between the victim and a remote server. Once this position is established, the attacker is able to eavesdrop on the communication using packet capture tools such as tcpdump.

Conceptual Overview

A Man-in-the-Middle attack allows an attacker to intercept and observe communication between two endpoints without interrupting the data exchange.

This attack relies on three key components:

- ICMP redirect messages to manipulate routing behavior
- IP forwarding to maintain communication flow
- Packet capture to observe intercepted traffic

Methodology

Step 1 – Start a Server

On the destination host:

```
sudo docker exec -it csl-host-5 bash
```

```
nc -l -p 1234
```

This command starts a listening service that waits for incoming messages.

Step 2 – Generate Traffic from the Victim

On the victim machine:

```
sudo docker exec -it csl-victim bash
```

```
nc 192.168.60.5 1234
```

A message is sent to the server to create real traffic in the network.

Additionally, continuous traffic can be generated using:

ping 192.168.60.5

This ensures that ICMP redirect messages are processed by the victim.

Step 3 – Launch the ICMP Redirect Attack

On the attacker machine:

```
sudo docker exec -it csl-attacker bash
```

```
python3 icmp_spoof.py
```

The script sends spoofed ICMP redirect messages, impersonating the legitimate router and instructing the victim to use the malicious router as the new gateway.

Step 4 – Capture Traffic

On the malicious router:

```
sudo docker exec -it csl-malicious-router bash
```

```
tcpdump -i eth0 -n port 1234
```

This command captures traffic passing through the router, allowing observation of the communication between the victim and the server.

```
csl-malicious-router> tcpdump -i eth0 -n port 1234
tcpdump: verbose output suppressed, use -v or -vv for full protocol decode
listening on eth0, link-type EN10MB (Ethernet), capture size 262144 bytes
09:43:04.178521 IP 10.9.0.5.41740 > 192.168.60.5.1234: Flags [S], seq 2911411376, win 64240, options
[mss 1460,sackOK,TS val 3757011789 ecr 0,nop,wscale 7], length 0
09:43:04.198137 IP 10.9.0.5.41740 > 192.168.60.5.1234: Flags [S], seq 2911411376, win 64240, options
[mss 1460,sackOK,TS val 3757011789 ecr 0,nop,wscale 7], length 0
09:43:04.198883 IP 10.9.0.5.41740 > 192.168.60.5.1234: Flags [.], ack 1652066560, win 502, options [n
op,nop,TS val 3757011809 ecr 140679613], length 0
09:43:04.200921 IP 10.9.0.5.41740 > 192.168.60.5.1234: Flags [.], ack 1, win 502, options [nop,nop,TS
val 3757011809 ecr 140679613], length 0
09:43:05.983459 IP 10.9.0.5.41740 > 192.168.60.5.1234: Flags [P.], seq 0:5, ack 1, win 502, options [
nop,nop,TS val 3757013594 ecr 140679613], length 5
09:43:05.983956 IP 10.9.0.5.41740 > 192.168.60.5.1234: Flags [P.], seq 0:5, ack 1, win 502, options [
nop,nop,TS val 3757013594 ecr 140679613], length 5
09:43:17.616336 IP 10.9.0.5.41740 > 192.168.60.5.1234: Flags [P.], seq 5:10, ack 1, win 502, options
[nop,nop,TS val 3757025226 ecr 140681398], length 5
09:43:17.616785 IP 10.9.0.5.41740 > 192.168.60.5.1234: Flags [P.], seq 5:10, ack 1, win 502, options
[nop,nop,TS val 3757025226 ecr 140681398], length 5
]
```

Observation

The captured traffic shows packets such as:

IP 10.9.0.5 → 192.168.60.5

This confirms that the traffic is being routed through the malicious router.

Traffic Flow

After the attack is successful, the communication path becomes:

Victim → Malicious Router → Router → Server

Why the Attack Works

The success of the attack depends on several conditions:

- The victim accepts ICMP redirect messages
- The attacker spoofs the IP address of the legitimate router
- The malicious router is reachable within the same subnet
- IP forwarding is enabled on the malicious router

IP forwarding is essential, as it allows packets to be relayed to their destination without disrupting communication.

Routing Behavior

Routing Table

ip route

The routing table contains static routing information and remains unchanged during the attack.

Routing Cache

The routing cache stores temporary routing decisions and is influenced by ICMP redirect messages. The victim updates its routing behavior dynamically based on this cache.

Stability of the Attack

The attack may not always appear stable due to:

- The temporary nature of ICMP redirect messages
- Dependence on ongoing traffic

- Periodic refresh of routing decisions

Payload Considerations

The effectiveness of ICMP redirect messages depends on the payload structure.

An incorrect payload:

original = IP(...) / ICMP()

may be ignored because it does not correspond to real traffic.

A correct payload:

original = IP(...) / ICMP(type=8)

matches real ICMP echo requests (ping) and is therefore accepted by the victim.

Scapy Behavior

The Scapy library may automatically modify certain packet fields such as checksums and lengths. As a result, the packet constructed in code may differ from the packet actually transmitted, which can affect the success of the attack.

Proof of Successful MITM

The attack is considered successful when:

- Traffic passes through the malicious router
- Communication between victim and server remains intact
- Captured packets confirm interception

Conclusion

By exploiting ICMP redirect messages, the attacker successfully manipulates the routing behavior of the victim and positions the malicious router in the communication path. This allows interception of traffic without interrupting the communication.

The attack demonstrates how routing protocols can be abused to compromise confidentiality, while still preserving the integrity and availability of the communication channel.

Task 4: Modifying Messages

Objective

The objective of this task is to extend the previously established Man-in-the-Middle (MITM) attack by modifying the content of the intercepted traffic. While previous tasks focused on breaking confidentiality, this task demonstrates how integrity can also be compromised by altering packets in transit.

Approach

To achieve this, a Python script was developed using the `NetfilterQueue` and `Scapy` libraries. The script intercepts packets forwarded by the malicious router, inspects their payload, and modifies specific patterns before forwarding them to their destination.

The script provides a command-line interface with the following parameters:

- `--src`: source IP address of the traffic to modify (mandatory)
- `-f` / `--find`: pattern to search for
- `-r` / `--replace`: replacement string

Example execution:

```
python3 modify.py --src 10.9.0.5 -f "hello" -r "HELLO"
```

Implementation Overview

The script performs the following operations:

1. Captures packets using `NetfilterQueue`
2. Filters packets based on the source IP
3. Checks if the packet contains application data (Raw layer)
4. Searches for a specific pattern in the payload
5. Replaces the pattern if found
6. Recalculates checksums and forwards the packet

Execution Procedure

Step 1 – Start the Modification Script

On the malicious router:

```
sudo docker exec -it csl-malicious-router bash  
python3 modify.py --src 10.9.0.5 -f "hello" -r "HELLO"
```

The script waits for packets to process.

Step 2 – Redirect Traffic to User Space

In a second terminal on the malicious router:

```
sudo docker exec -it csl-malicious-router bash  
iptables -I FORWARD -j NFQUEUE --queue-num 0
```

This rule redirects forwarded packets to the Python script.

Step 3 – Generate Traffic

On the server:

```
docker exec -it csl-host-5 bash  
nc -l -p 1234
```

On the victim:

```
sudo docker exec -it csl-victim bash  
nc 192.168.60.5 1234
```

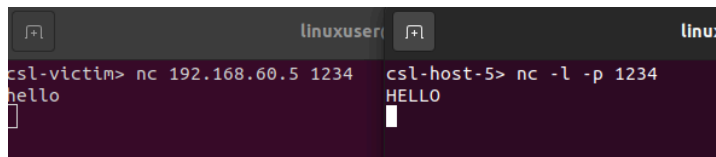
Send the message:

hello

Observed Result

The message received by the server is:

HELLO



This confirms that the packet was successfully intercepted and modified in transit.

Why No Destination Parameter Is Needed

The script does not require a `--dst` parameter because:

- All relevant traffic is already passing through the malicious router
- Filtering by source IP is sufficient to identify the targeted data stream
- The router processes packets regardless of their destination

Technical Considerations

1. Packet Modification Constraints

Replacing content with a different length may cause issues:

- TCP sequence numbers may become inconsistent
- Packet size changes may break the connection
- Some protocols expect fixed-length payloads

To avoid these problems, replacements with equal length are recommended.

2. Packet Visibility in Routers

By default, routers do not pass forwarded packets to user space applications. Therefore, the following configuration is required:

```
iptables -I FORWARD -j NFQUEUE --queue-num 0
```

This rule ensures that packets are intercepted before being forwarded.

3. Importance of Checksum Recalculation

After modifying a packet, checksums must be removed:

```
del pkt[IP].len  
del pkt[IP].chksum  
del pkt[TCP].chksum
```

The kernel automatically recalculates correct values. Without this step, packets would be rejected.

4. Performance Considerations

The script must process packets quickly to avoid:

- noticeable delays
- packet loss
- connection instability

Efficient filtering and minimal processing are therefore important.

Proof of Successful Attack

The attack is considered successful when:

- The victim sends data normally
- The server receives modified content
- Communication remains uninterrupted

Conclusion

This task demonstrates that a Man-in-the-Middle attack can compromise not only confidentiality but also data integrity. By intercepting and modifying packets in transit, the attacker can alter communication without detection.

The attack relies on the combination of:

- ICMP redirect to position the attacker
- IP forwarding to maintain communication
- Packet interception and modification via NetfilterQueue

This highlights the importance of secure routing mechanisms and validation of network control messages.